This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Learned Image Compression with Hierarchical Progressive Context Modeling

Yuqi Li* Haotian Zhang* Li Li Dong Liu
MOE Key Laboratory of Brain-Inspired Intelligent Perception and Cognition
University of Science and Technology of China, Hefei 230093, China
{lyq010303, zhanghaotian}@mail.ustc.edu.cn, {lil1, dongeliu}@ustc.edu.cn

Abstract

Context modeling is essential in learned image compression for accurately estimating the distribution of latents. While recent advanced methods have expanded context modeling capacity, they still struggle to efficiently exploit long-range dependency and diverse context information across different coding steps. In this paper, we introduce a novel Hierarchical Progressive Context Model (HPCM) for more efficient context information acquisition. Specifically, HPCM employs a hierarchical coding schedule to sequentially model the contextual dependencies among latents at multiple scales, which enables more efficient long-range context modeling. Furthermore, we propose a progressive context fusion mechanism that incorporates contextual information from previous coding steps into the current step, effectively exploiting diverse contextual information. Experimental results demonstrate that our method achieves state-of-the-art rate-distortion performance and strikes a better balance between compression performance and computational complexity. The code is available at https://github.com/lyq133/LIC-HPCM.

1. Introduction

Image compression is a fundamental topic in the field of computer vision and image processing, which plays a crucial role in enabling efficient visual data storage and transmission. In the passing decades, many traditional codecs have achieved impressive compression performance with manual design for each module, including JPEG [44], JPEG2000 [42], BPG [4], and VVC [6].

In recent years, various learned compression methods [15, 18, 23, 34] have demonstrated superior performance. Most learned image compression models are based on the transform coding scheme [12]. In such a model, the image is transformed to produce quantized latent representations. Subsequently, the quantized latents are entropy-coded into bitstreams through an entropy model. The decoder recovers the quantized latents and then applies a synthesis transform to reconstruct the image. The entropy model is used to estimate the distribution of latent variables for entropy coding. A more accurate entropy model leads to fewer bits, thereby improving compression performance.

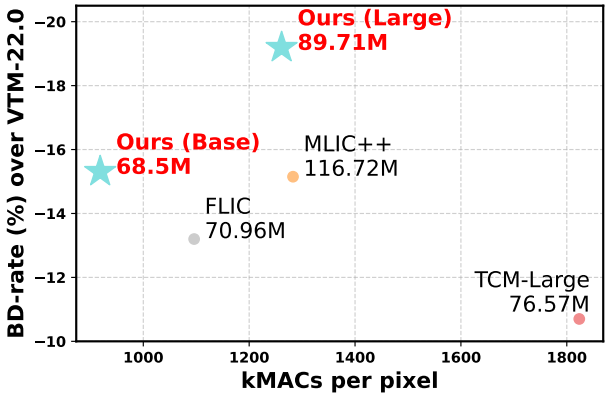


Figure 1. Comparison between BD-rate over VTM-22.0 and end-to-end computational complexity on Kodak dataset. We select recent advanced learned image compression methods [18, 23, 34] for comparison. Model parameters of various methods are noted in this figure. Top-left is better.

Most advanced learned image compression models apply conditional entropy models for entropy coding, which jointly utilize the hyperprior model [3] and context models [38, 39]. The context model divides latents into several groups and utilizes sequential autoregression to capture contextual information, thereby enabling accurate distribution estimation. Some studies utilized spatial context modeling [14, 39] to exploit the local spatial redundancy in the latent domain. Recent studies [40] have explored long-range spatial context modeling with transformer architectures, while introducing higher complexity. Some studies [14, 25] exploited context information in both channel and spatial dimensions. Jiang *et al.* [18] introduced

*Equal contribution. This work is supported by the Natural Science Foundation of China under Grant 62021001. We acknowledge the support of GPU cluster built by MCC Lab of Information Science and Technology Institution, USTC. (Corresponding author: Dong Liu.)

a multi-reference context model to effectively capture local spatial, global spatial, and channel-wise contextual information. While expanding the diversity of contextual information can boost compression performance, existing approaches still struggle to efficiently exploit long-range dependency and diverse context across different coding steps. Therefore, a more efficient method for exploiting diverse contexts is required to achieve a better trade-off between compression performance and computational complexity.

In this paper, we introduce a Hierarchical Progressive Context Model (HPCM) for more efficient context information acquisition. Our HPCM employs a hierarchical coding schedule to sequentially model contextual dependencies in latents at multiple scales. First, we partition the latents into multi-scale sub-latents through a specialized sampling method. Then, we sequentially code each scale of sub-latents, starting from the smallest scale and progressing to the largest, gradually modeling dependencies from long-range to short-range. Capturing context representations at a smaller scale of sub-latents enables more efficient modeling of long-range contextual dependencies. Furthermore, to better exploit the diverse context information at different coding steps, we present a progressive context fusion mechanism. Specifically, we incorporate the context information from previous coding steps into the current step's contextual representations through a cross-attention mechanism. This approach enables the progressive accumulation of diverse context information. Equipped with this progressive context fusion, our HPCM enables the efficient combination of diverse contexts at different coding steps.

To better balance compression performance and complexity, we introduce structural improvements and a parameter-efficient context model design, and optimize the arithmetic coding implementation. With the help of the proposed HPCM and model improvement, our method achieves state-of-the-art rate-distortion performance. As shown in Fig. 1, compared to VTM-22.0, our Base model and Large model achieve 15.31% and 19.19% rate savings on the Kodak dataset, respectively. Meanwhile, our methods strike a better balance between compression performance and computational complexity. Our contributions can be summarized as follows:

- We present a hierarchical coding schedule for learned image compression, gradually modeling dependencies in the latent domain from long-range to short-range. This enables more efficient long-range context modeling.
- We propose a progressive context fusion mechanism to enable the accumulation of diverse context, which incorporates the context from previous coding steps into the current step's contextual representations.
- Our method achieves state-of-the-art compression performance and strikes a better balance between compression performance and complexity.

2. Related Work

2.1. Learned Image Compression

In recent years, learned compression methods [16, 17, 24–26, 29–32, 43] have demonstrated impressive performance. Through the end-to-end optimization techniques and deep neural networks, learned image compression methods [15, 18, 23, 34] have achieved impressive performance. Most recent studies follow the joint rate-distortion optimization scheme [2] for advanced compression performance.

The capacity of transform is important for learned image compression. A convolution network-based transform is proposed in [2]. Subsequent studies [8, 9, 15, 52] utilized more powerful network structures to enhance transforms, such as deep residual connections and non-local attention mechanisms. The transformer-based transforms [23, 34, 51] are constructed to enable the global modeling ability. Invertible neural networks have also been explored in [36].

The entropy model is used to estimate the probabilistic distribution of the latents. In the conditional entropy model, the distribution of latents is modeled as a specific probability distribution family, such as the Gaussian model and generalized Gaussian model [48]. The hyperprior and context models are used to estimate the entropy parameters. For example, in [3], hyperprior is used for estimating the Gaussian scale parameters for characterizing the distribution of latents. Numerous studies have contributed to developing the context models, which are introduced in Sec. 2.2.

Some studies enhance the performance of learned image compression models through advancing the quantization strategies [13, 47]. Some studies aims to explore practical learned image compression models [28, 45, 50].

2.2. Context Models

The context model is effective for exploring the redundancies in latents. Specifically, it divides latents into several groups and utilizes sequential autoregression to capture contextual information, thereby enabling accurate distribution estimation. The raster-scan autoregressive spatial context model [39] is first adopted to exploit the local spatial redundancy in latents. Qian *et al.* [40] further explored long-range spatial context modeling with transformer architectures, while introducing higher complexity. He *et al.* [14] proposed a checkerboard model for parallel spatial context modeling. From another perspective, Minnen *et al.* [38] proposed an efficient channel-wise context model. Some studies [15, 25] exploited context information in both channel and spatial dimensions. Jiang *et al.* [18] introduced a multi-reference context model to capture local spatial, global spatial, and channel-wise contextual information. Kim *et al.* [19] utilized the diverse local, regional, and global contexts. While expanding the diversity of contextual information can boost compression performance, it

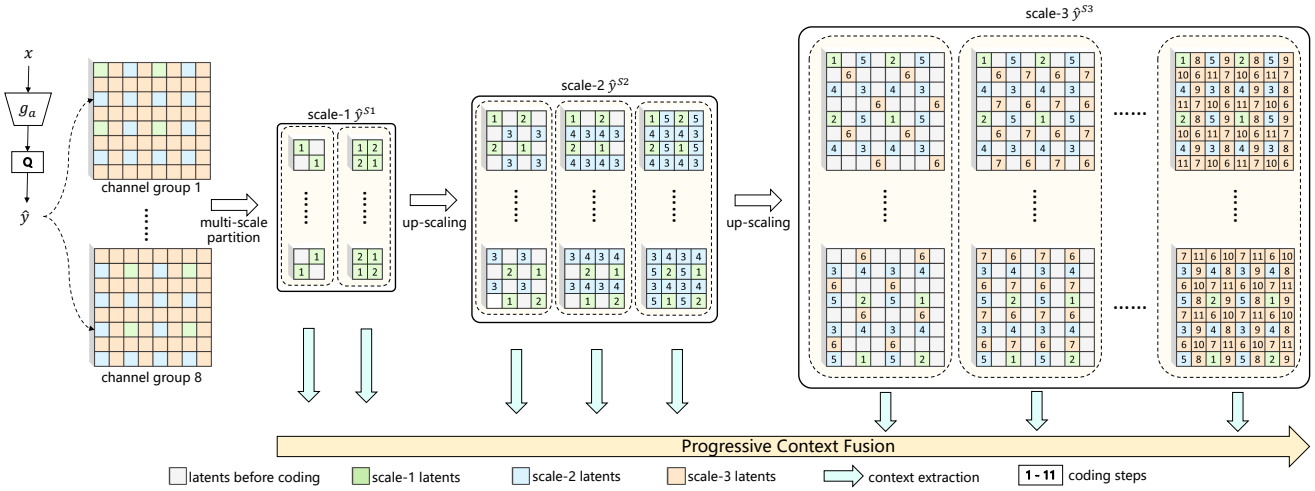


Figure 2. Hierarchical coding schedule of HPCM. $\hat{y}$ is partitioned into three sub-latents at different scales: $\hat{y}^{S1}$, $\hat{y}^{S2}$, and $\hat{y}^{S3}$. These partition strategies vary across different channel groups. We show the partition process of the first and eighth channel groups in this figure. Sequential conditional entropy coding and progressive context modeling are then applied at each coding step. We begin the coding process on $\hat{y}^{S1}$, capturing long-range dependency first. Once coding $\hat{y}^{S1}$, we fill $\hat{y}^{S1}$ back to the corresponding positions in $\hat{y}^{S2}$ for up-scaling. Similarly, after coding $\hat{y}^{S2}$, we fill $\hat{y}^{S2}$ into $\hat{y}^{S3}$. The coding steps at each scale are allocated based on the number of latent elements in each scale. The coding step for each element is noted in this figure. There are 11 coding steps in total.

also significantly increases the complexity of existing approaches. Therefore, a more efficient method for capturing diverse context is required to achieve a better trade-off between compression performance and complexity.

3. Proposed Methods

3.1. Overview

We follow the typical learned image compression scheme in our method. The encoder applies an analysis transform g_a to the input image $x \in \mathbb{R}^{3\times H\times W}$, generating latent representations $y = g_a(x|\phi) \in \mathbb{R}^{C\times H\times W}$. The latents are then rounded to obtain $\hat{y} = Q(y)$. The quantized latents are then losslessly entropy-coded through an entropy model $q_{\hat{Y}}(\hat{y})$. Finally, the decoder recovers $\hat{y}$ and generate reconstruction $\hat{x}$ through a synthesis transform $\hat{x} = g_s(\hat{y}|\theta)$. The notations ϕ and θ are trainable parameters of the analysis transform and synthesis transform, respectively.

Following the previous study [48], we model the distribution of $\hat{y}$ as a generalized Gaussian model $\mathcal{N}_\beta(\mu, \alpha)$ with the shape parameter β fixed as 1.5. The entropy model outputs the mean and scale parameters of the latents. We jointly use a hyperprior module and the proposed hierarchical progressive context model (HPCM) to estimate the entropy parameters of $\hat{y}$. The hyperprior module extracts side information z to capture redundancy in latents. The side information is obtained through a hyper-analysis transform $z = h_a(y|\phi_h)$, which is then quantized $\hat{z} = Q(z)$. The quantized side information is used to estimate the entropy parameters through a hyper-synthesis transform $h_s(\hat{z}|\theta_h)$. The notation ϕ_h and θ_h are trainable parameters of the hyper-analysis and hyper-synthesis transform, respectively. Our HPCM divides latents into several groups and codes each group sequentially. The entropy parameters for latents to be coded in the i_{th} coding step are obtained through

$$\mu_i, \alpha_i = \mathrm{HPCM}(\hat{y}_{<i}, h_s(\hat{z}|\theta_h)) \tag{1}$$

where $\hat{y}_{<i}$ denotes previously coded latents before i_{th} coding step.

Our HPCM employs a hierarchical coding schedule to sequentially model contextual dependencies among latents at multiple scales. First, we partition the latents into multi-scale sub-latents through a specialized sampling method. Then, we code each scale of sub-latents sequentially, starting from the smallest scale and progressing to the largest, gradually modeling dependencies from long-range to short-range. This hierarchical coding schedule is introduced in Sec. 3.2. Furthermore, to better exploit the diverse context information at different coding stages, we present a progressive context fusion mechanism. Specifically, we incorporate the context from previous coding steps into the current step's contextual representations through a cross-attention mechanism. This approach enables the progressive accumulation of diverse context information. This progressive context fusion is introduced in Sec. 3.3.

For end-to-end training, we optimize the model with the rate-distortion cost

$$L = \mathcal{R}(\hat{y}) + \mathcal{R}(\hat{z}) + \lambda \cdot \mathcal{D}(x, \hat{x}) \tag{2}$$

where $\mathcal{R}(\hat{y})$ and $\mathcal{R}(\hat{z})$ denote the bitrates of $\hat{y}$ and $\hat{z}$; $\mathcal{D}(x, \hat{x})$ denotes the distortion between x and $\hat{x}$; λ controls the rate-distortion tradeoff.

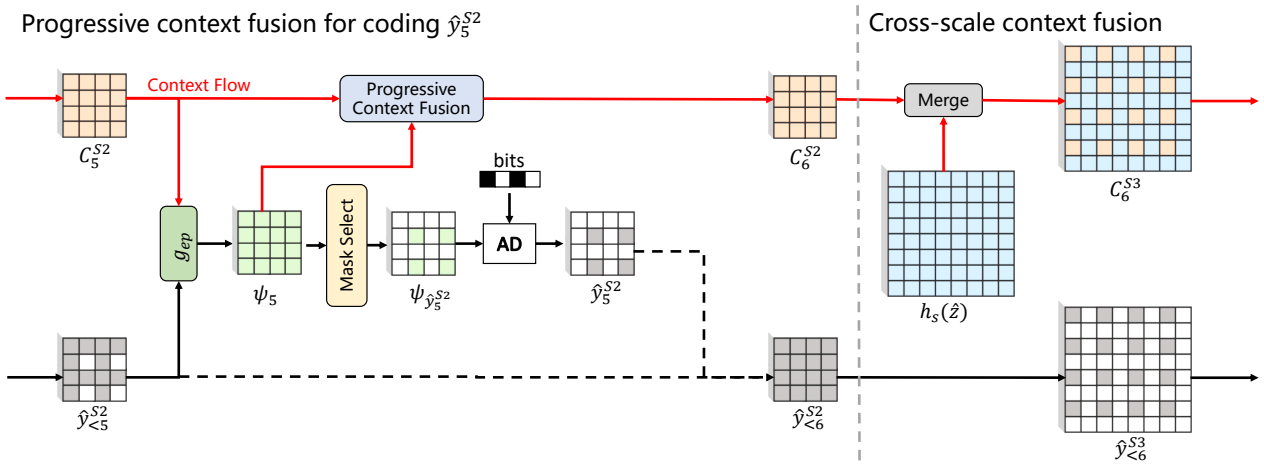


Figure 3. Diagram of our proposed Progressive Context Fusion (PCF) method. We take the coding process of $\hat{y}_5$ in $\hat{y}^{S2}$ and $\hat{y}_6$ in $\hat{y}^{S3}$ for example. C_i^{S2}, ψ_i and $\psi_{\hat{y}_i}$ denote the progressively accumulated context at S2, entropy parameter state, and the entropy parameters for coding $\hat{y}_i$, respectively. g_{ep} represents the entropy parameter network. The detailed process of the PCF module is shown in Fig. 4. In the cross-scale scenario, C_6^{S2} and $h_s(\hat{z})$ are merged to obtain C_6^{S3} at higher scale.

3.2. Hierarchical Coding Schedule

Utilizing long-range context information is crucial for more accurate distribution estimation. Some learned image compression methods [22, 40] have exploited long-range context information in the latent domain through global attention mechanisms, demonstrating advanced compression performance. However, these methods also introduce higher complexity due to explicitly modeling long-range dependencies in the latent domain. Our HPCM addresses this limitation through a hierarchical coding schedule. As illustrated in Fig. 2, we partition $\hat{y}$ into three sub-latents at different scales: $\hat{y}^{S1}$, $\hat{y}^{S2}$, and $\hat{y}^{S3}$. Coding smaller sub-latents first and then capturing context information on them enables a larger spatial receptive field, thereby facilitating more efficient long-range context modeling. Additionally, following the joint spatial-channel context model [25], we apply different spatial partition strategies on eight evenly sliced channel groups. This allows for the interaction of spatial and channel-wise context information. The coding process of two different channel groups is shown in Fig. 2 to illustrate our hierarchical coding schedule.

We apply sequential conditional entropy coding at each scale and progressively exploit context information in each coding step. Specifically, $\hat{y}^{S1}$ is a 4× down-scaled version of $\hat{y}$. The first entropy coding step for $\hat{y}^{S1}$ is conditioned only on the hyperprior information. After completing the first coding step, subsequent steps are conditioned on our HPCM. Once $\hat{y}^{S1}$ is coded, we fill it back into its corresponding locations in $\hat{y}^{S2}$. Similarly, after $\hat{y}^{S2}$ has been coded, we fill it back into its corresponding locations in $\hat{y}^{S3}$. This up-scaling process, from $\hat{y}^{S1}$ to $\hat{y}^{S3}$, gradually models dependencies from long-range to short-range, ensuring comprehensive context representation across all scales.

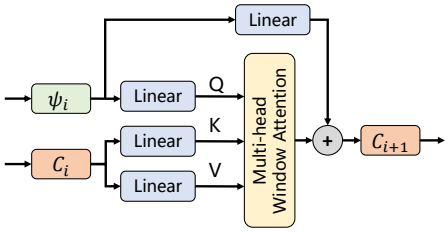


Figure 4. Illustration of Progressive Context Fusion (PCF) module. C_i and ψ_i denote the progressively accumulated context and entropy parameter state, respectively.

To achieve a better balance between compression performance and complexity, we allocate different coding steps at each scale. Specifically, for $\hat{y}^{S1}$, the smallest scale, we employ an efficient 2-step coding process. This allows us to maintain low computational complexity while still capturing essential long-range context information. As the scale increases, the number of latents increases, requiring finer context modeling to save bits. We apply a 3-step quadtree-based entropy coding [25] for $\hat{y}^{S2}$ and a 6-step octree-based entropy coding for $\hat{y}^{S3}$. We follow previous studies [27, 37] to design the octree-based coding order at $\hat{y}^{S3}$. Specifically, within each 4×4 patch on the spatial dimension, we initially code latent elements widely dispersed across the spatial dimension. Subsequent coded latents are systematically positioned adjacent to previously coded latents. This approach exploits the spatial correlation in latents and enhances the overall context modeling ability. The detailed coding schedules are included in Sec. C of the supplement material.

3.3. Progressive Context Fusion

Previous studies usually extract context reference from the hyperprior and the previously coded latents for entropy

modeling. To better exploit the diverse contexts at different coding steps, our proposed progressive context fusion module incorporates the context information from previous coding steps into the current step's context reference. For the first coding step, the context information is down-scaled from the synthesized hyperprior. For the following coding steps, the context information C_i is accumulated from previous coding steps. This approach aims to progressively accumulate diverse context information for more efficient utilization of context information for entropy coding.

We take the coding for $\hat{y}_i^{S2}$ to illustrate the progressive fusion process. At i_{th} coding step, the context information C_i^{S2} is generated by fusing the context information C_{i-1}^{S2} and the entropy parameter state ψ_{i-1} at $i-1_{th}$ coding step. The entropy parameter state ψ_{i-1} contains the entropy parameters used for coding $\hat{y}_{i-1}^{S2}$. As illustrated in Fig. 3, the entropy parameter state ψ_i in i_{th} step is updated through

$$\psi_i = g_{ep}(\text{Concat}(\hat{y}_{<i}^{S2}, C_i^{S2})) \tag{3}$$

where g_{ep} denotes the entropy parameter network. Then the corresponding entropy parameters $\psi_{\hat{y}_i^{S2}}$ for coding $\hat{y}_i^{S2}$ are selected from ψ_i. Similarly, C_{i+1}^{S2} is obtained through combining C_i^{S2} and ψ_i for progressive context accumulation. As show in Fig. 4, C_i^{S2} and ψ_i are fused through a cross-attention module. This fusion process can be formulated by

$$Q = \text{Linear}(\psi_i), \tag{4}$$

$$K = \text{Linear}(C_i^{S2}), \tag{5}$$

$$V = \text{Linear}(C_i^{S2}), \tag{6}$$

$$C_{i+1}^{S2} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V + \text{Linear}(\psi_i), \tag{7}$$

where d_k is the dimension of K.

Additionally, to efficiently propagate the context information across multiple scales, we further propose a cross-scale context fusion method. We take the context propagation process from $\hat{y}^{S2}$ to $\hat{y}^{S3}$ to illustrate this cross-scale context fusion. As shown in Fig. 3, to propagate the context information C_6^{S2} at smaller scale to the context C_6^{S3} at larger scale, we fill C_6^{S2} to the corresponding locations in C_6^{S3}. For the remaining locations, we fill the synthesized hyperprior information $h_s(\hat{z})$. This approach efficiently combines the hyperprior information and cross-scale context information.

Through this progressive context fusion at each coding step, the accumulated context C_i progressively exhibits enhanced contextual diversity. Moreover, with the context fusion across scales, this progressively refined C_i also enables efficient context fusion of both long-range and short-range dependencies in the hierarchical coding process, strengthening the entropy estimation capability.

3.4. Other Improvements

First, we conduct improvements in network structures in our model to strike a better balance between compression performance and complexity. Specifically, we utilized the advanced design of neural networks [7, 10, 49, 50] to construct transforms and context model networks. Additionally, we present a parameter-efficient way to construct context models, which shares the same network at different coding stages [25]. This design significantly reduces model parameters while preserving essential context-aware adaptation capabilities. More details on the network structure are introduced in Sec. A and Sec. B of the supplement material. Second, we optimized the entropy coder implementation through more efficient data exchange between Python and C, which enables efficient entropy coding.

4. Experimental Results

4.1. Experimental Settings

Training settings. We use Flicker2W dataset [33] for training. During training, images are randomly cropped to 256 × 256 patches with a batch size of 32. The models are optimized by Eq. (2). When optimized for mean square error (MSE) distortion metric, the Lagrange multiplier λ belongs to $\{0.0018, 0.0035, 0.0067, 0.0130, 0.0250, 0.0483\}$. When optimized for multi-scale structural similarity (MS-SSIM) [46], λ belongs to $\{2.40, 4.58, 8.73, 16.64, 31.73, 60.50\}$. The Adam [20] optimizer is used with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. Our models are optimized with 2 million training steps. The learning rate is set as 10^{-4} initially, and reduced to 2×10^{-5} after 1.6M steps, then to 5×10^{-6} after 1.8M steps, and then to 10^{-6} after 1.9M steps.

Evaluation settings. We evaluate performance on three commonly used test datasets: Kodak dataset [21] which contains 24 images with 512 × 768 resolution; CLIC Professional Validation (CLIC Pro Valid) dataset [1] which contains 41 high-quality images; Tecnick dataset [1] which contains 100 images with 1200 × 1200 resolution. We use bits-per-pixel (bpp) to measure the bitrate and Peak Signal-to-Noise Ratio (PSNR) or MS-SSIM to measure the distortion. We convert MS-SSIM to $-10\log_{10}(1 - \text{MS-SSIM})$ for better comparison of rate-distortion curves. We also use the BD-Rate metric [5] to evaluate rate saving. The anchor of BD-Rate in this paper is VTM-22.0[2]. The coding time is evaluated on a single-core Intel(R) Xeon(R) Gold 6248R CPU and an NVIDIA GeForce RTX 3090 GPU. The model complexities, including kMACs/pixel and model parameter counts, are evaluated with DeepSpeed library[3].

[1] http://compression.cc
[2] https://vcgit.hhi.fraunhofer.de/jvet/VVCSoftware_VTM
[3] https://github.com/microsoft/DeepSpeed

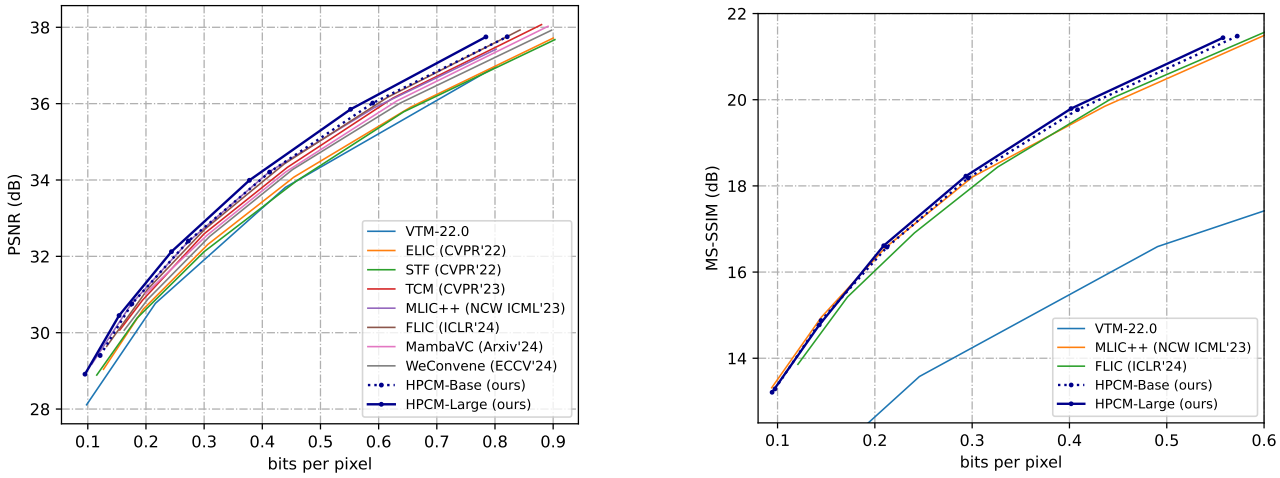


Figure 5. Rate-distortion curves on Kodak dataset.

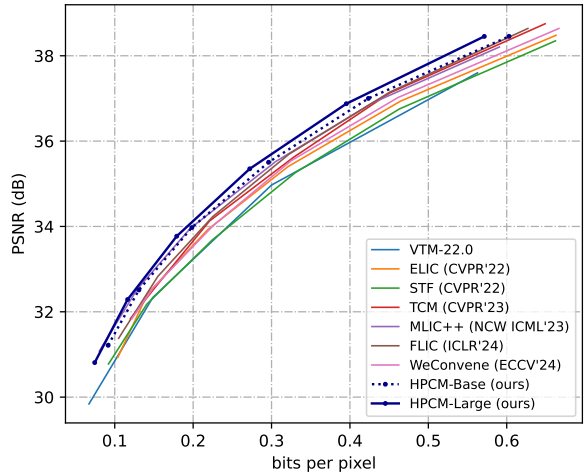


Figure 6. Rate-distortion curves on CLIC Pro Valid dataset.

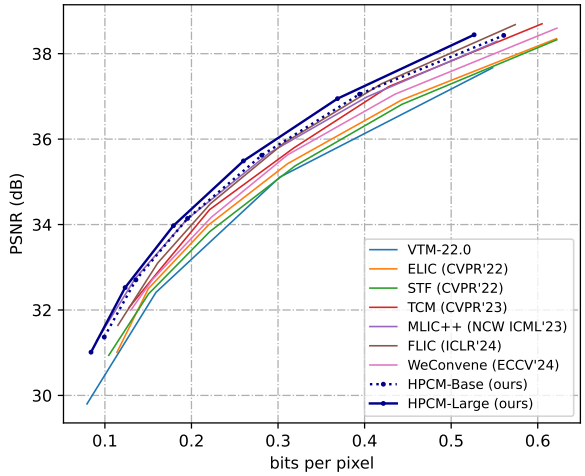


Figure 7. Rate-distortion curves on Tecnick dataset.

Model settings. We present HPCM-Base and HPCM-Large targeting different complexities. The difference between the two models lies in the number of network layers in transform and entropy parameter network. The details are included in Sec. A and Sec. B of the supplement material.

4.2. Rate-Distortion Performance

First, we verify the effectiveness of our proposed HPCM. Specifically, we train the CHARM [38] and DCVC-DC intra [25] entropy models using the same transform architecture as our HPCM-Base model. As shown in Table 1, our proposed method achieves significantly higher compression performance, demonstrating the effectiveness of the proposed HPCM.

Then, we compare our proposed two models, HPCM-Base and HPCM-Large, to state-of-the-art (SOTA) learned image compression methods, including [11, 15, 18, 23, 34, 41, 52]. As shown in Fig. 5, our HPCM-Base model achieves better performance compared to other advanced methods on both PSNR and MS-SSIM metrics. Our HPCM-Large model achieves higher performance, showing about 0.2dB higher PSNR compared to other advanced methods at the same bitrate. Fig. 6 and Fig. 7 demonstrate our superior performance on the CLIC Pro Valid and Tecnick datasets. Table 1 shows the BD-Rate performance of our methods. Compared to VTM-22.0, our HPCM-Large model achieves 19.19%, 18.37%, and 22.20% BD-Rate gain on Kodak, CLIC Pro Valid, and Tecnick datasets, respectively.

4.3. Complexity

Table 1 shows the model complexity of our method and various SOTA learned image compression methods, including encoding and decoding times, kMACs/pixel, and model parameters. Benefiting from our efficient hierarchical progressive context modeling and improved network architecture, our method achieves a better trade-off between compression performance and complexity. Compared to MLIC++, our

Table 1. Compression performance and complexity comparison. VTM-22.0 is used as an anchor to calculate the PSNR BD-Rate. The best compression performance is marked in **bold**, and the second-best is underlined.

Model	Enc. Time† (ms)	Dec. Time† (ms)	kMACs /pixel	Params (M)	PSNR BD-Rate		
					Kodak	CLIC Pro Valid	Tecnick
ELIC (CVPR'22) [15]	126.5	111.4	573.88	36.93	-3.22%	-3.89%	-4.57%
STF (CVPR'22) [52]	142.5	156.8	511.17	99.86	-2.06%	1.12%	-2.17%
TCM (CVPR'23) [34]	200.2	201.8	1823.58	76.57	-10.70%	-8.32%	-11.84%
MLIC++ (NCW ICML'23) [18]	193.4	226.4	1282.81	116.72	-15.15%	-14.05%	-17.90%
FLIC (ICLR'24) [23]	>1000	>1000	1096.04	70.96	-13.20%	-9.88%	-15.27%
MambaVC (Arxiv'24) [41]	235.6	246.2	813.80	47.88	-8.72%	-	-
WeConvene (ECCV'24) [11]	343.6	256.5	2343.13	107.15	-6.98%	-5.66%	-8.63%
CHARM*	57.5	70.6	495.75	58.53	0.86%	1.55%	-1.32%
DCVC-DC intra*	57.8	58.2	542.14	45.51	-9.18%	-8.54%	-10.18%
HPCM-Base (ours)	81.8	81.3	918.57	68.50	-15.31%	-14.23%	-18.16%
HPCM-Large (ours)	91.2	90.2	1261.29	89.71	**-19.19%**	**-18.37%**	**-22.20%**

*The transforms are the same as our HPCM-Base model, and the entropy models are different.
† Coding time includes network inference time and arithmetic coding time. Details are presented in Sec. F of the supplement material.

Table 2. Abaltion studies on hierarchical coding schedule.

Model settings	kMACs/pixel	BD-Rate
HPCM-Base*	918.57	0.00%
w/o hierarchical extraction	1107.48	1.07%
coding step (2, 3, 3)	663.90	2.39%
coding step (2, 3, 12)	1427.91	-2.55%
coding step (4, 3, 6)	925.59	0.35%

* Our default setting is coding step (2, 3, 6).

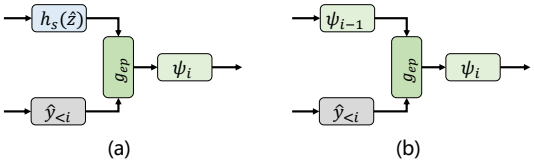


Figure 8. Illustration of methods to obtain the entropy parameter state ψ_i in Table 3. (a) Without progressive fusion. (b) Use ψ_i as progressive context.

Table 3. Abaltion studies on progressive context fusion.

Model settings	kMACs/pixel	BD-Rate
HPCM-Base	918.57	0.00%
w/o progressive fusion	872.80	4.71%
use ψ_i as progressive context	872.80	1.17%

HPCM-Base model achieves comparable performance with much lower kMACs/pixel, and our HPCM-Large model achieves significantly higher performance with comparable kMACs/pixel. As for the coding time, the total runtime reported in Table 1 includes both network inference time and arithmetic coding time. A detailed breakdown of coding time is provided in Sec. F of the supplement material.

4.4. Visual Comparison

Figure 9 visualizes reconstructed images of Kodim24 in the Kodak dataset with different learned image compression methods. In some specific texture regions, our methods can keep more details.

4.5. Ablation Studies

4.5.1. Ablation Studies on Hierarchical Coding Schedule

To show the effectiveness of our hierarchical coding schedule, we conduct ablations on our HPCM-Base model. First, we evaluate the model without the hierarchical context extraction, where the context information at different steps is exploited at the largest scale. As shown in Table 2, this model results in higher computational complexity due to context modeling at the original latent scale. Additionally, since exploiting context information at a smaller scale enables more efficient long-range context modeling, our hierarchical context extraction achieves better performance. More ablation studies on hierarchical coding stages are presented in Sec. G.1 of the supplement material.

We also show the influence of the number of coding steps allocated in different scales. In our HPCM model, the number of coding steps allocated for coding $(\hat{y}^{S1}, \hat{y}^{S2}, \hat{y}^{S3})$ is (2, 3, 6). Table 2 shows the performance of other settings. The detailed coding schedules of these settings are introduced in Sec. C of the supplement material. Since there are more latents at larger scales, the performance gain of using more coding steps at the largest scale is more significant. In contrast, at the smallest scale, increasing the number of coding steps has minimal influence. Although the model with coding step (2, 3, 12) further achieves 2.55% rate saving over HPCM-Base, the computational complexity is much higher. To achieve a better balance between compression performance and complexity, we adopt the coding step as (2, 3, 6) in our methods.

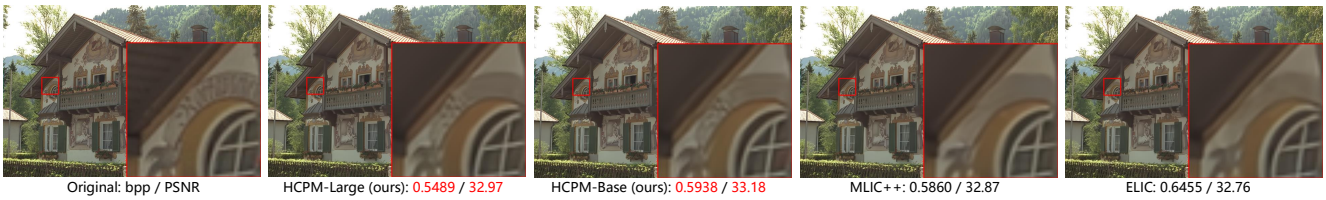


Figure 9. Visualization of reconstructed images of Kodim24 in the Kodak dataset with various learned image compression methods.

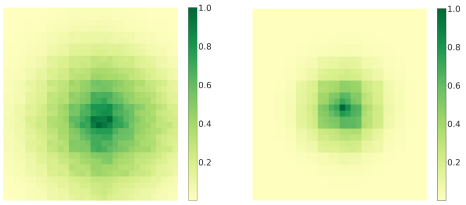


Figure 10. Effective receptive fields (ERF) [35] visualization of the entropy parameter network (a) g_{ep}^{S1} and (b) g_{ep}^{S3} in our HPCM-Base model. A wider dark area indicates a larger ERF.

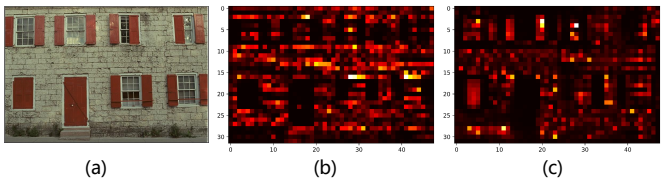


Figure 11. Visualization of (a) Original image Kodim01 from the Kodak dataset; (b) Bit allocation map of latent $\hat{y}_{<6}^{S3}$. (c) Attention map of the progressive context fusion module at 6_{th} coding step on $\hat{y}^{S3}$.

4.5.2. Ablation Studies on Progressive Context Fusion

In this section, we show the effectiveness of our proposed progressive context fusion method. In the model without progressive context fusion, as shown in Fig. 8 (a), we use the hyperprior $h_s(\hat{z})$ and previous coded latents $\hat{y}_{<i}$ to get ψ_i through the entropy parameter network g_{ep} at i_{th} coding step. This approach is widely used in recent learned image compression methods. As shown in Table 3, although the progressive fusion module introduces a slight increase in complexity, removing it results in a 4.71% performance drop, demonstrating the effectiveness of our progressive context fusion method.

We also compare different ways to achieve progressive context fusion. As illustrated in Fig. 8 (b), for comparison, we use ψ_{i-1} and $\hat{y}_{<i}$ to obtain ψ_i for entropy coding at i_{th} step. Since the entropy parameter state ψ_i retains context information from the previous $i-1$ coding steps, using ψ_i as the progressive accumulated context achieves comparable performance. However, introducing C_i as the progressive context information through a cross-attention mechanism yields better performance. While both methods leverage context information from previous coding steps, the cross-attention mechanism used with C_i in our methods allows for more precise context integration, leading to improved performance.

4.6. Visualization Analysis

Hierarchical Coding Schedule captures both long-range and short-range dependencies. Figure 10 visualizes the effective receptive field (ERF) [35] of entropy parameter networks in our HPCM-Base model at different scales: g_{ep}^{S1} and g_{ep}^{S3}. We visualize the gradient flow from the output of the entropy parameter networks to the input latents. The results are averaged on the Kodak dataset. The visualizations show that g_{ep}^{S1} exhibits a relatively large ERF, while the ERF of g_{ep}^{S3} is smaller, indicating that context extraction at $\hat{y}^{S1}$ captures long-range global patterns, while context extraction at $\hat{y}^{S3}$ focuses on short-range detailed patterns.

Cross-attention in Progressive Context Fusion (PCF) module focuses more on high-bitrate regions. Figure 11 visualizes the bit allocation map of $\hat{y}_{<6}^{S3}$ and the attention map of the cross-attention operation in the PCF module at 6_{th} coding step on $\hat{y}^{S3}$ in our HPCM-Base model. For visualizing, we select the center point as the query in each window attention and rearrange attention maps to match the original latent size.

The bit allocation map in Fig. 11 (b) shows that our model allocates higher bitrates to regions with more complex textures, which are more challenging to encode. The attention map in Fig. 11 (c) also assigns larger values to these high-bitrate regions. This indicates that our PCF module progressively leads C_i to focus more on these high-bitrate regions that are more challenging for context modeling, thereby improving the accuracy of the entropy model.

5. Conclusion

In this paper, we propose a novel hierarchical progressive context model for more efficient entropy coding in learned image compression. First, we introduce a hierarchical coding schedule to efficiently enable long-range context utilization. Furthermore, we propose a progressive context fusion module, which incorporates contextual information from previous coding steps and progressively accumulates the context information to efficiently enhance context diversity. Experimental results demonstrate that our proposed method achieves state-of-the-art performance and has a better trade-off between compression performance and complexity.

References

[1] Nicola Asuni and Andrea Giachetti. TESTIMAGES: A large-scale archive for testing visual devices and basic image processing algorithms. In *Smart Tools and Apps for Graphics - Eurographics Italian Chapter Conference*, pages 63–70, 2014. 5

[2] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations (ICLR)*, 2017. 2

[3] Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. Variational image compression with a scale hyperprior. In *International Conference on Learning Representations (ICLR)*, 2018. 1, 2

[4] F. Bellard. BPG Image Format. http://bellard.org/bpg/, 2015. 1

[5] Gisle Bjontegaard. Calculation of average psnr differences between rd-curves. *ITU SG16 Doc. VCEG-M33*, 2001. 5

[6] Benjamin Bross, Ye-Kui Wang, Yan Ye, Shan Liu, Jianle Chen, Gary J. Sullivan, and Jens-Rainer Ohm. Overview of the versatile video coding (VVC) standard and its applications. *IEEE Transactions on Circuits and Systems for Video Technology*, 31(10):3736–3764, 2021. 1

[7] Jierun Chen, Shiu-hong Kao, Hao He, Weipeng Zhuo, Song Wen, Chul-Ho Lee, and S.-H. Gary Chan. Run, don't walk: Chasing higher FLOPS for faster neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12021–12031, 2023. 5

[8] Tong Chen, Haojie Liu, Zhan Ma, Qiu Shen, Xun Cao, and Yao Wang. End-to-end learnt image compression via non-local attention optimization and improved context modeling. *IEEE Transactions on Image Processing*, 30:3179–3191, 2021. 2

[9] Zhengxue Cheng, Heming Sun, Masaru Takeuchi, and Jiro Katto. Learned image compression with discretized Gaussian mixture likelihoods and attention modules. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7936–7945, 2020. 2

[10] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1800–1807, 2017. 5

[11] Haisheng Fu, Jie Liang, Zhenman Fang, Jingning Han, Feng Liang, and Guohe Zhang. Weconvene: Learned image compression with wavelet-domain convolution and entropy model. In *European Conference on Computer Vision (ECCV)*, pages 37–53, 2024. 6, 7

[12] V.K. Goyal. Theoretical foundations of transform coding. *IEEE Signal Processing Magazine*, 18(5):9–21, 2001. 1

[13] Zongyu Guo, Zhizheng Zhang, Runsen Feng, and Zhibo Chen. Soft then hard: Rethinking the quantization in neural image compression. In *International Conference on Machine Learning (ICML)*, pages 3920–3929, 2021. 2

[14] Dailan He, Yaoyan Zheng, Baocheng Sun, Yan Wang, and Hongwei Qin. Checkerboard context model for efficient learned image compression. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14766–14775, 2021. 1, 2

[15] Dailan He, Ziming Yang, Weikun Peng, Rui Ma, Hongwei Qin, and Yan Wang. ELIC: Efficient learned image compression with unevenly grouped space-channel contextual adaptive coding. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5708–5717, 2022. 1, 2, 6, 7

[16] Zhihao Hu, Guo Lu, and Dong Xu. Fvc: A new framework towards deep video compression in feature space. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1502–1511, 2021. 2

[17] Zhaoyang Jia, Bin Li, Jiahao Li, Wenxuan Xie, Linfeng Qi, Houqiang Li, and Yan Lu. Towards practical real-time neural video compression. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2025, Nashville, TN, USA, June 11-25, 2024*, 2025. 2

[18] Wei Jiang and Ronggang Wang. MLIC++: Linear complexity multi-reference entropy modeling for learned image compression. In *ICML Workshop*, 2023. 1, 2, 6, 7

[19] Jun-Hyuk Kim, Seungeon Kim, Won-Hee Lee, and Dokwan Oh. Diversify, contextualize, and adapt: Efficient entropy modeling for neural image codec. pages 45956–45974, 2024. 2

[20] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. 5

[21] Eastman Kodak. Kodak lossless true color image suite. http://r0k.us/graphics/kodak/. 5

[22] Daxin Li, Yuanchao Bai, Kai Wang, Junjun Jiang, Xianming Liu, and Wen Gao. GroupedMixer: An entropy model with group-wise token-mixers for learned image compression. *IEEE Transactions on Circuits and Systems for Video Technology*, 34(10):9606–9619, 2024. 4

[23] Han Li, Shaohui Li, Wenrui Dai, Chenglin Li, Junni Zou, and Hongkai Xiong. Frequency-aware transformer for learned image compression. In *International Conference on Learning Representations (ICLR)*, 2024. 1, 2, 6, 7

[24] Jiahao Li, Bin Li, and Yan Lu. Deep contextual video compression. *Advances in Neural Information Processing Systems*, 34, 2021. 2

[25] Jiahao Li, Bin Li, and Yan Lu. Neural video compression with diverse contexts. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22616–22626, 2023. 1, 2, 4, 5, 6

[26] Jiahao Li, Bin Li, and Yan Lu. Neural video compression with feature modulation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 17-21, 2024*, 2024. 2

[27] Yuqi Li, Haotian Zhang, and Dong Liu. Flexible coding order for learned image compression. In *IEEE International Conference on Visual Communications and Image Processing (VCIP)*, pages 1–5, 2023. 4

[28] Yuqi Li, Haotian Zhang, Xiaomin Song, Zheng Liu, Huiming Zheng, Li Li, and Dong Liu. Deviation control for learned image compression. In *2024 IEEE International Conference on Visual Communications and Image Processing (VCIP)*, pages 1–5, 2024. 2

[29] Zhuoyuan Li, Jiacheng Li, Yao Li, Li Li, Dong Liu, and Feng Wu. In-loop filtering via trained look-up tables. In *2024*

IEEE International Conference on Visual Communications and Image Processing (VCIP), pages 1–5. IEEE, 2024. 2

[30] Zhuoyuan Li, Yao Li, Chuanbo Tang, Li Li, Dong Liu, and Feng Wu. Uniformly accelerated motion model for inter prediction. In *2024 IEEE International Conference on Visual Communications and Image Processing (VCIP)*, pages 1–5. IEEE, 2024.

[31] Zhuoyuan Li, Zikun Yuan, Li Li, Dong Liu, Xiaohu Tang, and Feng Wu. Object segmentation-assisted inter prediction for versatile video coding. *IEEE Transactions on Broadcasting*, 2024.

[32] Zhuoyuan Li, Junqi Liao, Chuanbo Tang, Haotian Zhang, Yuqi Li, Yifan Bian, Xihua Sheng, Xinmin Feng, Yao Li, Changsheng Gao, et al. USTC-TD: A test dataset and benchmark for image and video coding in 2020s. *IEEE Transactions on Multimedia*, 2025. 2

[33] Jiaheng Liu, Guo Lu, Zhihao Hu, and Dong Xu. A unified end-to-end framework for efficient deep image compression. *arXiv preprint arXiv:2002.03370*, 2020. 5

[34] Jinming Liu, Heming Sun, and Jiro Katto. Learned image compression with mixed Transformer-CNN architectures. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14388–14397, 2023. 1, 2, 6, 7

[35] Wenjie Luo, Yujia Li, Raquel Urtasun, and Richard Zemel. Understanding the effective receptive field in deep convolutional neural networks. 2016. 8

[36] Haichuan Ma, Dong Liu, Ning Yan, Houqiang Li, and Feng Wu. End-to-end optimized versatile image compression with wavelet-like transform. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3):1247–1263, 2022. 2

[37] Fabian Mentzer, Eirikur Agustson, and Michael Tschannen. M2T: Masking Transformers twice for faster decoding. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5317–5326, 2023. 4

[38] David Minnen and Saurabh Singh. Channel-wise autoregressive entropy models for learned image compression. In *IEEE International Conference on Image Processing (ICIP)*, pages 3339–3343, 2020. 1, 2, 6

[39] David Minnen, Johannes Ballé, and George Toderici. Joint autoregressive and hierarchical priors for learned image compression. In *Advances in Neural Information Processing Systems*, pages 10794–10803, 2018. 1, 2

[40] Yichen Qian, Ming Lin, Xiuyu Sun, Zhiyu Tan, and Rong Jin. Entroformer: A Transformer-based entropy model for learned image compression. In *International Conference on Learning Representations (ICLR)*, 2022. 1, 2, 4

[41] Shiyu Qin, Jinpeng Wang, Yiming Zhou, Bin Chen, Tianci Luo, Baoyi An, Tao Dai, Shutao Xia, and Yaowei Wang. Mambavc: Learned visual compression with selective state spaces. *arXiv preprint arXiv:2405.15413*, 2024. 6, 7

[42] A. Skodras, C. Christopoulos, and T. Ebrahimi. The JPEG 2000 still image compression standard. *IEEE Signal Processing Magazine*, 18(5):36–58, 2001. 1

[43] Chuanbo Tang, Zhuoyuan Li, Yifan Bian, Li Li, and Dong Liu. Neural video compression with context modulation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 12553–12563, 2025. 2

[44] Gregory K Wallace. The JPEG still picture compression standard. *Communications of the ACM*, 34(4):30–44, 1991. 1

[45] Shen Wang, Zhengxue Cheng, Donghui Feng, Guo Lu, Li Song, and Wenjun Zhang. Asymllic: Asymmetric lightweight learned image compression. In *2024 IEEE International Conference on Visual Communications and Image Processing (VCIP)*, pages 1–5, 2024. 2

[46] Zhou Wang, Eero P Simoncelli, and Alan C Bovik. Multiscale structural similarity for image quality assessment. In *The Thrity-Seventh Asilomar Conference on Signals, Systems & Computers, 2003*, pages 1398–1402. Ieee, 2003. 5

[47] Haotian Zhang, Li Li, and Dong Liu. On uniform scalar quantization for learned image compression. *arXiv preprint arXiv:2309.17051*, 2023. 2

[48] Haotian Zhang, Li Li, and Dong Liu. Generalized Gaussian model for learned image compression. *arXiv preprint arXiv:2411.19320*, 2024. 2, 3

[49] Haotian Zhang, Feihong Mei, Junqi Liao, Li Li, Houqiang Li, and Dong Liu. Practical learned image compression with online encoder optimization. In *Picture Coding Symposium (PCS)*, pages 1–5, 2024. 5

[50] Haotian Zhang, Yuqi Li, Li Li, and Dong Liu. Learning switchable priors for neural image compression. *IEEE Transactions on Circuits and Systems for Video Technology*, 2025. 2, 5

[51] Yinhao Zhu, Yang Yang, and Taco Cohen. Transformer-based transform coding. In *International Conference on Learning Representations (ICLR)*, 2022. 2

[52] Renjie Zou, Chunfeng Song, and Zhaoxiang Zhang. The devil is in the details: Window-based attention for image compression. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17471–17480, 2022. 2, 6, 7